

Guía de Actividades Práctico-Experimentales Nro. 005

1. Datos Generales

Estudiantes	Vasquez Calderon Jostin Xavier Martina Alejandra Maldonado Machuca
Asignatura	Estructura de datos
Ciclo	3 A
Unidad	2
Resultado de aprendizaje de la unidad	Aplica los métodos de ordenación y búsqueda en la resolución de problemas, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Título de la Práctica	Ordenación básica en Java: Burbuja, Selección e Inserción
Nombre del Docente	Andrés Roberto Navas Castellanos
Fecha	Jueves 13 de noviembre Viernes 14 de noviembre
Horario	07h30 – 10h30 07h30 – 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	5 horas

2. Objetivo(s) de la Práctica:

- Implementar y comparar tres algoritmos de ordenación in-place sobre arreglos pequeños, y validar su funcionamiento con trazas y casos de prueba reproducibles.

3. Materiales y reactivos:

- Guía de pruebas con datasets y salidas esperadas.

4. Equipos y herramientas

- JDK OpenJDK (obligatorio).
- IDE: Visual Studio Code (extensión "Extension Pack for Java") o IntelliJ IDEA Community.
- Sistema de control de versiones: Git; repositorio en GitHub.
- EVA/Moodle institucional: para entrega de evidencias.
- Herramientas de documentación: README Markdown, editor ofimático (Google Docs/LibreOffice/Word).

5. Procedimiento / Metodología

Enfoque metodológico: ABPr (Aprendizaje Basado en Proyectos). Inicio

- Presentación breve de objetivos y alcance (qué implementar y cómo verificar).
- Organización individual o en parejas; crear repo Git y rama de trabajo.
- Revisión de firmas y criterios de evaluación (rúbrica).
- Lineamientos de uso responsable de IA.

Desarrollo

- Implementar en este orden: Inserción, Selección, Burbuja (facilita comparar).

- **Estructura del proyecto**

- Paquete ed.u2.sorting.
- Clases sugeridas: InsertionSort, SelectionSort, BubbleSort, SortingDemo, SortingUtils.

- **Firmas estándar (Java)**

```
public final class InsertionSort {  
    public static void sort(int[] a) { /* ... */ }  
}  
public final class SelectionSort {  
    public static void sort(int[] a) { /* ... */ }  
}  
public final class BubbleSort {  
    public static void sort(int[] a) { /* ... */ }  
}
```

- Todas **in-place** (modifican el arreglo recibido).
- Agregar sobrecarga opcional sort(int[] a, boolean trace) para imprimir trazas.
- **Inserción (recomendado empezar aquí)**
 - Bucle externo i=1..n-1; insertar a[i] en 0..i-1.
 - **Trazas**: imprimir posiciones movidas y arreglo resultante por iteración.
- **Selección**
 - Para cada i, hallar **mínimo** en i..n-1 y cambiar por a[i].
 - Contabilizar **swaps** (pocos intercambios).
- **Burbuja**
 - Pasadas sucesivas; intercambiar adyacentes "mal ordenados".
 - Optimización: corte temprano si no hubo **swaps** en una pasada.
- **Casos bordes**
 - Arreglo vacío, tamaño 1, todos iguales, ya ordenado, orden inverso, con duplicados.
- **Trazas y validación**
 - Ejecutar SortingDemo con datasets de la sección 10.
 - Guardar capturas/salidas en TXT para subir al EVA.

Cierre

- Comparación cualitativa: cuándo usar cada algoritmo (datos pequeños, casi ordenados, número de swaps, legibilidad de traza).
- Revisión de evidencias y dudas frecuentes.
- Recordatorio de entregables y modo de verificación automática en EVA.

6. Resultados esperados:

- Git con src/ (clases de sort y SortingDemo), y README.md (1 pág.: decisiones, casos borde, cómo ejecutar).

Repositorio: <https://github.com/xDylok/Ordenacion>

- Evidencias:

InsertionSorting

```
Seleccione una opcion:
  1. InsertionSorting
  2. SelectionSorting
  3. BubbleSorting
  4. Salir.
Opcion: 1

| ----- Insertion Sort ----- |
| ----- Data Set A: [8, 3, 6, 3, 9] ----- |

>Iteracion: 1, Insercion: 3
  Actual: [8, 3, 6, 3, 9]
    -Pasando: '8' de posicion: 0 hacia: 1
Resultado: [3, 8, 6, 3, 9]

>Iteracion: 2, Insercion: 6
  Actual: [3, 8, 6, 3, 9]
    -Pasando: '8' de posicion: 1 hacia: 2
Resultado: [3, 6, 8, 3, 9]

>Iteracion: 3, Insercion: 3
  Actual: [3, 6, 8, 3, 9]
    -Pasando: '8' de posicion: 2 hacia: 3
    -Pasando: '6' de posicion: 1 hacia: 2
Resultado: [3, 3, 6, 8, 9]
```

```
>Iteracion: 4, Insercion: 1
  Actual: [2, 3, 4, 5, 1]
    -Pasando: '5' de posicion: 3 hacia: 4
    -Pasando: '4' de posicion: 2 hacia: 3
    -Pasando: '3' de posicion: 1 hacia: 2
    -Pasando: '2' de posicion: 0 hacia: 1
Resultado: [1, 2, 3, 4, 5]
| ----- Resultado Final: [1, 2, 3, 4, 5] ----- |

| ----- Data Set C: [1, 2, 3, 4, 5] ----- |

>Iteracion: 1, Insercion: 2
  Actual: [1, 2, 3, 4, 5]
Resultado: [1, 2, 3, 4, 5]

>Iteracion: 2, Insercion: 3
  Actual: [1, 2, 3, 4, 5]
Resultado: [1, 2, 3, 4, 5]

>Iteracion: 3, Insercion: 4
  Actual: [1, 2, 3, 4, 5]
Resultado: [1, 2, 3, 4, 5]

>Iteracion: 4, Insercion: 5
  Actual: [1, 2, 3, 4, 5]
Resultado: [1, 2, 3, 4, 5]
| ----- Resultado Final: [1, 2, 3, 4, 5] ----- |

| ----- Data Set D: [2, 2, 2, 2] ----- |
```

```
>Iteracion: 1, Insercion: 2
  Actual: [2, 2, 2, 2]
Resultado: [2, 2, 2, 2]

>Iteracion: 2, Insercion: 2
  Actual: [2, 2, 2, 2]
Resultado: [2, 2, 2, 2]

>Iteracion: 3, Insercion: 2
  Actual: [2, 2, 2, 2]
Resultado: [2, 2, 2, 2]
| ----- Resultado Final: [2, 2, 2, 2] ----- |

| ----- Data Set E: [9, 1, 8, 2] ----- |

>Iteracion: 1, Insercion: 1
  Actual: [9, 1, 8, 2]
    -Pasando: '9' de posicion: 0 hacia: 1
Resultado: [1, 9, 8, 2]

>Iteracion: 2, Insercion: 8
  Actual: [1, 9, 8, 2]
    -Pasando: '9' de posicion: 1 hacia: 2
Resultado: [1, 8, 9, 2]

>Iteracion: 3, Insercion: 2
  Actual: [1, 8, 9, 2]
    -Pasando: '9' de posicion: 2 hacia: 3
    -Pasando: '8' de posicion: 1 hacia: 2
```

```
>Iteracion: 4, Insercion: -4
  Actual: [-10, -8, -5, -1, -4]
  -Pasando: '-1' de posicion: 3 hacia: 4
Resultado: [-10, -8, -5, -4, -1]
| ————— Resultado Final: [-10, -8, -5, -4, -1] ————— |

| ————— > CB: Data Set I: [0, -8, -3, 1, 3, -10] ————— |

>Iteracion: 1, Insercion: -8
  Actual: [0, -8, -3, 1, 3, -10]
  -Pasando: '0' de posicion: 0 hacia: 1
Resultado: [-8, 0, -3, 1, 3, -10]

>Iteracion: 2, Insercion: -3
  Actual: [-8, 0, -3, 1, 3, -10]
  -Pasando: '0' de posicion: 1 hacia: 2
Resultado: [-8, -3, 0, 1, 3, -10]

>Iteracion: 3, Insercion: 1
  Actual: [-8, -3, 0, 1, 3, -10]
Resultado: [-8, -3, 0, 1, 3, -10]

>Iteracion: 4, Insercion: 3
  Actual: [-8, -3, 0, 1, 3, -10]
Resultado: [-8, -3, 0, 1, 3, -10]

Resultado: [1, 2, 8, 9]
| ————— Resultado Final: [1, 2, 8, 9] ————— |

| ————— > CB: Data Set F: [] ————— |
| ————— Resultado Final: [] ————— |

| ————— > CB: Data Set G: [2] ————— |
| ————— Resultado Final: [2] ————— |

| ————— > CB: Data Set H: [-1, -10, -8, -5, -4] ————— |

>Iteracion: 1, Insercion: -10
  Actual: [-1, -10, -8, -5, -4]
  -Pasando: '-1' de posicion: 0 hacia: 1
Resultado: [-10, -1, -8, -5, -4]

>Iteracion: 2, Insercion: -8
  Actual: [-10, -1, -8, -5, -4]
  -Pasando: '-1' de posicion: 1 hacia: 2
Resultado: [-10, -8, -1, -5, -4]

>Iteracion: 3, Insercion: -5
  Actual: [-10, -8, -1, -5, -4]
  -Pasando: '-1' de posicion: 2 hacia: 3
Resultado: [-10, -8, -5, -1, -4]
```

```
>Iteracion: 5, Insercion: -10
  Actual: [-8, -3, 0, 1, 3, -10]
    -Pasando: '3' de posicion: 4 hacia: 5
    -Pasando: '1' de posicion: 3 hacia: 4
    -Pasando: '0' de posicion: 2 hacia: 3
    -Pasando: '-3' de posicion: 1 hacia: 2
    -Pasando: '-8' de posicion: 0 hacia: 1
Resultado: [-10, -8, -3, 0, 1, 3]
| ————— Resultado Final: [-10, -8, -3, 0, 1, 3] ————— |

| ————— > CB: Data Set J: [1, 3, 4, 10, 6] ————— |

>Iteracion: 1, Insercion: 3
  Actual: [1, 3, 4, 10, 6]
Resultado: [1, 3, 4, 10, 6]

>Iteracion: 2, Insercion: 4
  Actual: [1, 3, 4, 10, 6]
Resultado: [1, 3, 4, 10, 6]

>Iteracion: 3, Insercion: 10
  Actual: [1, 3, 4, 10, 6]
Resultado: [1, 3, 4, 10, 6]

>Iteracion: 4, Insercion: 6
  Actual: [1, 3, 4, 10, 6]
    -Pasando: '10' de posicion: 3 hacia: 4
Resultado: [1, 3, 4, 6, 10]
| ————— Resultado Final: [1, 3, 4, 6, 10] ————— |
```

```
| ————— > CB: Data Set K: [10, 1, 3, 4, 6] ————— |

>Iteracion: 1, Insercion: 1
  Actual: [10, 1, 3, 4, 6]
    -Pasando: '10' de posicion: 0 hacia: 1
Resultado: [1, 10, 3, 4, 6]

>Iteracion: 2, Insercion: 3
  Actual: [1, 10, 3, 4, 6]
    -Pasando: '10' de posicion: 1 hacia: 2
Resultado: [1, 3, 10, 4, 6]

>Iteracion: 3, Insercion: 4
  Actual: [1, 3, 10, 4, 6]
    -Pasando: '10' de posicion: 2 hacia: 3
Resultado: [1, 3, 4, 10, 6]

>Iteracion: 4, Insercion: 6
  Actual: [1, 3, 4, 10, 6]
    -Pasando: '10' de posicion: 3 hacia: 4
Resultado: [1, 3, 4, 6, 10]
| ————— Resultado Final: [1, 3, 4, 6, 10] ————— |
```

SelectionSorting

Opcion: 2

```
| ----- Selection Sort ----- |
| ----- Data Set A: [8, 3, 6, 3, 9] ----- |
| Iteracion: |0|: Min encontrado: |3| Arreglo actualizado: [3, 8, 6, 3, 9]
| Iteracion: |1|: Min encontrado: |3| Arreglo actualizado: [3, 3, 6, 8, 9]
| Iteracion: |2|: Min encontrado: |6| Arreglo actualizado: [3, 3, 6, 8, 9]
| Iteracion: |3|: Min encontrado: |8| Arreglo actualizado: [3, 3, 6, 8, 9]
Total de swaps: 2
| ----- Resultado Final: [3, 3, 6, 8, 9] ----- |

| ----- Data Set B: [5, 4, 3, 2, 1] ----- |
| Iteracion: |0|: Min encontrado: |1| Arreglo actualizado: [1, 4, 3, 2, 5]
| Iteracion: |1|: Min encontrado: |2| Arreglo actualizado: [1, 2, 3, 4, 5]
| Iteracion: |2|: Min encontrado: |3| Arreglo actualizado: [1, 2, 3, 4, 5]
| Iteracion: |3|: Min encontrado: |4| Arreglo actualizado: [1, 2, 3, 4, 5]
Total de swaps: 2
| ----- Resultado Final: [1, 2, 3, 4, 5] ----- |

| ----- Data Set C: [1, 2, 3, 4, 5] ----- |
| Iteracion: |0|: Min encontrado: |1| Arreglo actualizado: [1, 2, 3, 4, 5]
| Iteracion: |1|: Min encontrado: |2| Arreglo actualizado: [1, 2, 3, 4, 5]
| Iteracion: |2|: Min encontrado: |3| Arreglo actualizado: [1, 2, 3, 4, 5]
| Iteracion: |3|: Min encontrado: |4| Arreglo actualizado: [1, 2, 3, 4, 5]
Total de swaps: 0
| ----- Resultado Final: [1, 2, 3, 4, 5] ----- |
```

```
| ----- Data Set D: [2, 2, 2, 2] ----- |
| Iteracion: |0|: Min encontrado: |2| Arreglo actualizado: [2, 2, 2, 2]
| Iteracion: |1|: Min encontrado: |2| Arreglo actualizado: [2, 2, 2, 2]
| Iteracion: |2|: Min encontrado: |2| Arreglo actualizado: [2, 2, 2, 2]
Total de swaps: 0
| ----- Resultado Final: [2, 2, 2, 2] ----- |

| ----- Data Set E: [9, 1, 8, 2] ----- |
| Iteracion: |0|: Min encontrado: |1| Arreglo actualizado: [1, 9, 8, 2]
| Iteracion: |1|: Min encontrado: |2| Arreglo actualizado: [1, 2, 8, 9]
| Iteracion: |2|: Min encontrado: |8| Arreglo actualizado: [1, 2, 8, 9]
Total de swaps: 2
| ----- Resultado Final: [1, 2, 8, 9] ----- |

| ----- > CB: Data Set F: [] ----- |
Total de swaps: 0
| ----- Resultado Final: [] ----- |

| ----- > CB: Data Set G: [2] ----- |
Total de swaps: 0
| ----- Resultado Final: [2] ----- |

| ----- > CB: Data Set H: [-1, -10, -8, -5, -4] ----- |
| Iteracion: |0|: Min encontrado: |-10| Arreglo actualizado: [-10, -1, -8, -5, -4]
| Iteracion: |1|: Min encontrado: |-8| Arreglo actualizado: [-10, -8, -1, -5, -4]
| Iteracion: |2|: Min encontrado: |-5| Arreglo actualizado: [-10, -8, -5, -1, -4]
| Iteracion: |3|: Min encontrado: |-4| Arreglo actualizado: [-10, -8, -5, -4, -1]
Total de swaps: 4
| ----- Resultado Final: [-10, -8, -5, -4, -1] ----- |
```

```
| ----- > CB: Data Set I: [0, -8, -3, 1, 3, -10] ----- |
  Iteracion: |0|: Min encontrado: |-10| Arreglo actualizado: [-10, -8, -3, 1, 3, 0]
  Iteracion: |1|: Min encontrado: |-8| Arreglo actualizado: [-10, -8, -3, 1, 3, 0]
  Iteracion: |2|: Min encontrado: |-3| Arreglo actualizado: [-10, -8, -3, 1, 3, 0]
  Iteracion: |3|: Min encontrado: |0| Arreglo actualizado: [-10, -8, -3, 0, 3, 1]
  Iteracion: |4|: Min encontrado: |1| Arreglo actualizado: [-10, -8, -3, 0, 1, 3]
Total de swaps: 3
| ----- Resultado Final: [-10, -8, -3, 0, 1, 3] ----- |

| ----- > CB: Data Set J: [1, 3, 4, 10, 6] ----- |
  Iteracion: |0|: Min encontrado: |1| Arreglo actualizado: [1, 3, 4, 10, 6]
  Iteracion: |1|: Min encontrado: |3| Arreglo actualizado: [1, 3, 4, 10, 6]
  Iteracion: |2|: Min encontrado: |4| Arreglo actualizado: [1, 3, 4, 10, 6]
  Iteracion: |3|: Min encontrado: |6| Arreglo actualizado: [1, 3, 4, 6, 10]
Total de swaps: 1
| ----- Resultado Final: [1, 3, 4, 6, 10] ----- |

| ----- > CB: Data Set K: [10, 1, 3, 4, 6] ----- |
  Iteracion: |0|: Min encontrado: |1| Arreglo actualizado: [1, 10, 3, 4, 6]
  Iteracion: |1|: Min encontrado: |3| Arreglo actualizado: [1, 3, 10, 4, 6]
  Iteracion: |2|: Min encontrado: |4| Arreglo actualizado: [1, 3, 4, 10, 6]
  Iteracion: |3|: Min encontrado: |6| Arreglo actualizado: [1, 3, 4, 6, 10]
Total de swaps: 4
| ----- Resultado Final: [1, 3, 4, 6, 10] ----- |
```

BubbleSort

```
Opcion: 3

| ----- Bubble Sort ----- |
| ----- Data Set A: [8, 3, 6, 3, 9] ----- |

-Pasada: 1
Actual: [8, 3, 6, 3, 9]
  > SWAP: 8 ↔ 3 | [3, 8, 6, 3, 9]
  > SWAP: 8 ↔ 6 | [3, 6, 8, 3, 9]
  > SWAP: 8 ↔ 3 | [3, 6, 3, 8, 9]

-Pasada: 2
Actual: [3, 6, 3, 8, 9]
  > SWAP: 6 ↔ 3 | [3, 3, 6, 8, 9]

-Pasada: 3
Actual: [3, 3, 6, 8, 9]
Corte Temprano: no hubo intercambios. Array ordenado.
| ----- Resultado Final: [3, 3, 6, 8, 9] ----- |

| ----- Data Set B: [5, 4, 3, 2, 1] ----- |

-Pasada: 1
Actual: [5, 4, 3, 2, 1]
  > SWAP: 5 ↔ 4 | [4, 5, 3, 2, 1]
  > SWAP: 5 ↔ 3 | [4, 3, 5, 2, 1]
  > SWAP: 5 ↔ 2 | [4, 3, 2, 5, 1]
  > SWAP: 5 ↔ 1 | [4, 3, 2, 1, 5]
```



```
-Pasada: 2
Actual: [4, 3, 2, 1, 5]
  > SWAP: 4 ↔ 3 | [3, 4, 2, 1, 5]
  > SWAP: 4 ↔ 2 | [3, 2, 4, 1, 5]
  > SWAP: 4 ↔ 1 | [3, 2, 1, 4, 5]

-Pasada: 3
Actual: [3, 2, 1, 4, 5]
  > SWAP: 3 ↔ 2 | [2, 3, 1, 4, 5]
  > SWAP: 3 ↔ 1 | [2, 1, 3, 4, 5]

-Pasada: 4
Actual: [2, 1, 3, 4, 5]
  > SWAP: 2 ↔ 1 | [1, 2, 3, 4, 5]

-Pasada: 5
Actual: [1, 2, 3, 4, 5]
Corte Temprano: no hubo intercambios. Array ordenado.
| ————— Resultado Final: [1, 2, 3, 4, 5] ————— |

| ————— Data Set C: [1, 2, 3, 4, 5] ————— |

-Pasada: 1
Actual: [1, 2, 3, 4, 5]
Corte Temprano: no hubo intercambios. Array ordenado.
| ————— Resultado Final: [1, 2, 3, 4, 5] ————— |
```

```
| ————— Data Set D: [2, 2, 2, 2] ————— |

-Pasada: 1
Actual: [2, 2, 2, 2]
Corte Temprano: no hubo intercambios. Array ordenado.
| ————— Resultado Final: [2, 2, 2, 2] ————— |

| ————— Data Set E: [9, 1, 8, 2] ————— |

-Pasada: 1
Actual: [9, 1, 8, 2]
  > SWAP: 9 ↔ 1 | [1, 9, 8, 2]
  > SWAP: 9 ↔ 8 | [1, 8, 9, 2]
  > SWAP: 9 ↔ 2 | [1, 8, 2, 9]

-Pasada: 2
Actual: [1, 8, 2, 9]
  > SWAP: 8 ↔ 2 | [1, 2, 8, 9]

-Pasada: 3
Actual: [1, 2, 8, 9]
Corte Temprano: no hubo intercambios. Array ordenado.
| ————— Resultado Final: [1, 2, 8, 9] ————— |
```

```
| _____ > CB: Data Set F: [] _____ |  
| _____ Resultado Final: [] _____ |
```

```
| _____ > CB: Data Set G: [2] _____ |  
| _____ Resultado Final: [2] _____ |
```

```
| _____ > CB: Data Set H: [-1, -10, -8, -5, -4] _____ |
```

-Pasada: 1

Actual: [-1, -10, -8, -5, -4]

> SWAP: -1 $\leftrightarrow$ -10 | [-10, -1, -8, -5, -4]

> SWAP: -1 $\leftrightarrow$ -8 | [-10, -8, -1, -5, -4]

> SWAP: -1 $\leftrightarrow$ -5 | [-10, -8, -5, -1, -4]

> SWAP: -1 $\leftrightarrow$ -4 | [-10, -8, -5, -4, -1]

-Pasada: 2

Actual: [-10, -8, -5, -4, -1]

Corte Temprano: no hubo intercambios. Array ordenado.

```
| _____ Resultado Final: [-10, -8, -5, -4, -1] _____ |
```

```
| _____ > CB: Data Set I: [0, -8, -3, 1, 3, -10] _____ |
```

-Pasada: 1

Actual: [0, -8, -3, 1, 3, -10]

> SWAP: 0 $\leftrightarrow$ -8 | [-8, 0, -3, 1, 3, -10]

> SWAP: 0 $\leftrightarrow$ -3 | [-8, -3, 0, 1, 3, -10]

> SWAP: 3 $\leftrightarrow$ -10 | [-8, -3, 0, 1, -10, 3]

-Pasada: 2

Actual: [-8, -3, 0, 1, -10, 3]

> SWAP: 1 $\leftrightarrow$ -10 | [-8, -3, 0, -10, 1, 3]

-Pasada: 3

Actual: [-8, -3, 0, -10, 1, 3]

> SWAP: 0 $\leftrightarrow$ -10 | [-8, -3, -10, 0, 1, 3]

-Pasada: 4

Actual: [-8, -3, -10, 0, 1, 3]

> SWAP: -3 $\leftrightarrow$ -10 | [-8, -10, -3, 0, 1, 3]

-Pasada: 5

Actual: [-8, -10, -3, 0, 1, 3]

> SWAP: -8 $\leftrightarrow$ -10 | [-10, -8, -3, 0, 1, 3]

-Pasada: 6

Actual: [-10, -8, -3, 0, 1, 3]

Corte Temprano: no hubo intercambios. Array ordenado.

```
| _____ Resultado Final: [-10, -8, -3, 0, 1, 3] _____ |
```

```
| ————— > CB: Data Set J: [1, 3, 4, 10, 6] ————— |

-Pasada: 1
Actual: [1, 3, 4, 10, 6]
  > SWAP: 10 ↔ 6 | [1, 3, 4, 6, 10]

-Pasada: 2
Actual: [1, 3, 4, 6, 10]
Corte Temprano: no hubo intercambios. Array ordenado.
| ————— Resultado Final: [1, 3, 4, 6, 10] ————— |

| ————— > CB: Data Set K: [10, 1, 3, 4, 6] ————— |

-Pasada: 1
Actual: [10, 1, 3, 4, 6]
  > SWAP: 10 ↔ 1 | [1, 10, 3, 4, 6]
  > SWAP: 10 ↔ 3 | [1, 3, 10, 4, 6]
  > SWAP: 10 ↔ 4 | [1, 3, 4, 10, 6]
  > SWAP: 10 ↔ 6 | [1, 3, 4, 6, 10]

-Pasada: 2
Actual: [1, 3, 4, 6, 10]
Corte Temprano: no hubo intercambios. Array ordenado.
| ————— Resultado Final: [1, 3, 4, 6, 10] ————— |
```

- Trazas/outputs (.txt o capturas) de los datasets requeridos.

Archivo Trazas.txt se encuentra subido en el repositorio.

- Tabla breve comparando recuentos de movimientos/intercambios observados (cualitativo).

Algoritmo	Comparaciones	Swaps	Cuando usar
Insertion	Pocos si está casi ordenado	Muchos ya que desplaza muchos elementos	Datos pequeños o casi ordenados.
Selection	Muchas ya que compara todo	Pocos, uno por pasada, máximo: n-1	Cuando hacer los swaps es muy costoso
Bubble	Muchas ya que compara todo	Muchos	Para aprendizaje ya que es muy lento



unl

Universidad
Nacional
de Loja

7. Preguntas de Control:

Datasets y salidas esperadas:

- $A = [8, 3, 6, 3, 9]$
- $B = [5, 4, 3, 2, 1]$ (inverso)
- $C = [1, 2, 3, 4, 5]$ (ya ordenado)
- $D = [2, 2, 2, 2]$ (duplicados)
- $E = [9, 1, 8, 2]$

Orden final esperado (para los tres algoritmos)

- $A \rightarrow [3, 3, 6, 8, 9]$
- $B \rightarrow [1, 2, 3, 4, 5]$
- $C \rightarrow [1, 2, 3, 4, 5]$
- $D \rightarrow [2, 2, 2, 2]$
- $E \rightarrow [1, 2, 8, 9]$

Preguntas de control:

- **¿Por qué la Inserción es preferible con datos casi ordenados?**

El método de Inserción funciona muy bien cuando los datos ya están casi en orden porque avanza comparando cada elemento solo con los que están inmediatamente antes. Si detecta que un elemento ya está en su posición correcta, continúa sin necesidad de mover muchos valores. Por esta razón, en listas casi ordenadas avanza muy rápido y realiza muy pocos movimientos, siendo más eficiente que Burbuja o Selección.

- **¿Qué propiedad hace que la Selección use pocos swaps? ¿Qué compromisos tiene?**

La propiedad que hace que use pocos intercambios o swaps es que no intercambia elementos cada vez que encuentra uno más pequeño. Este solo lee todo el arreglo y selecciona el índice del elemento mínimo (min) durante toda una pasada. Al final de esa pasada, realiza un único intercambio (si $\text{min} \neq i$) para poner ese mínimo en su lugar.

- **¿Cómo implementarías el corte temprano en Burbuja y qué caso reduce significativamente?**

En nuestro caso, se implementó una variable "boolean 'swapped' = false" la cual se reinicia al inicio de cada pasada. Si se realiza un intercambio, a la variable se le asigna un valor 'true'. Al final de cada pasada se comprueba si con la condición 'if' implementada. Esto reduce el caso del array que ya está ordenado. En lugar de hacer $n-1$ pasadas, hará una sola pasada, comprobará que swapped sigue en 'false' y terminará.

- **¿Cuál(es) de los tres puede(n) ser estable y en qué condiciones?**

Entre los tres algoritmos, Bubble Sort e Insertion Sort son inherentemente estables debido a su diseño: Bubble Sort mantiene la estabilidad porque solo intercambia elementos adyacentes cuando son estrictamente mayores, preservando el orden relativo de elementos iguales, mientras que Insertion Sort la garantiza al insertar elementos iguales después de los existentes durante los desplazamientos; por el contrario, Selection Sort no es estable por defecto ya que los intercambios a distancia pueden alterar el orden relativo, aunque podría modificarse para ser estable mediante desplazamientos en lugar de swaps, pero esto eliminaría su principal ventaja de realizar pocos intercambios.

- **Menciona dos casos bordes que deben probarse siempre.**

Dos casos borde esenciales que siempre deben probarse son: el array vacío para verificar que el algoritmo maneje correctamente tamaño cero y el array con un solo elemento confirmando que lo considere trivialmente ordenado; adicionalmente, son cruciales el array ya ordenado para evaluar el mejor caso, el array inversamente ordenado (analizando el peor caso) y arrays con elementos duplicados (fundamental para validar la estabilidad y correcto manejo de iguales).

8. Evaluación

Criterio	4 – Excelente	3 – Bueno	2 – Básico	1 – Insuficiente	Pts
Implementación InsertionSort	Correcta; traza clara; maneja casos borde	Detalles menores	Parcial	No funcional	3.0
Implementación SelectionSort	Correcta; contabiliza swaps	Detalles menores	Parcial	No funcional	2.5
Implementación BubbleSort	Correcta; corte temprano implementado	Detalles menores	Parcial	No funcional	2.5
Evidencias (README + salidas)	Completas, reproducibles	Aceptables	Escasas	Inexistentes	1.0
Calidad de código	Nombres/estructura/comentarios adecuados	Aceptable	Pobre	Deficiente	1.0

9. Bibliografía

- [1] OpenDSA Project, "Sorting and Searching Modules," Virginia Tech, 2021–2024 (REA con visualizaciones y ejercicios).
- [2] P. W. Bible and L. Moser, An Open Guide to Data Structures and Algorithms. PALNI Open Press, 2023.
- [3] Oracle, "Java SE 17–21 Documentation: Arrays, Collections, and I/O (java.nio.file)." 2021–2025.
- [4] T. Roughgarden, Algorithms Illuminated (Part 1), Soundlikeyourself Publishing, 2019–2020 (con recursos online actualizados).

10. Elaboración y Aprobación

Elaborado por	Andrés R Navas Castellanos Docente	 Firmado electrónicamente por: ANDRES ROBERTO NAVAS CASTELLANOS Validar únicamente con FirmaEC
Revisado por Solo si es realizado en laboratorios	Luis Sinche Técnico Docente	No Aplica
Aprobado por	Edison L Coronel Romero Director de Carrera	 Firmado electrónicamente por: EDISON LEONARDO CORONEL ROMERO Validar únicamente con FirmaEC